



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**DataQuest: Sistema Web de Validación de
Normalización de Bases de Datos Relacionales**

Curso: Base de Datos II

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Perez Peralta, Fabrizio Salvador Elias
Dongo Palza, Manuel Andree

(2023077476)
(2023076842)

Tacna - Perú
2026

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Equipo DataQuest Analista BD Frontend Backend/API			19/06/2026	Versión Original

DataQuest: Sistema Web de Validación de Normalización de Bases de Datos Relacionales

Informe de Factibilidad

Versión 1.0

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Equipo DataQuest Analista BD Frontend Backend/API			19/06/2026	Versión Original

ÍNDICE GENERAL

1. Descripción del Proyecto.....	5
1.1. Nombre del proyecto	5
1.2. Duración del proyecto	5
1.3. Descripción.....	5
1.4. Objetivos	6
1.4.1. Objetivo general	6
1.4.2. Objetivos Específicos	6
2. Riesgos.....	8
3. Análisis de la Situación actual	8
3.1. Planteamiento del problema	8
3.2. Descripción del árbol de problema.....	9
3.2.1. Procesos manuales de validación	10
3.2.2. Dependencia del conocimiento individual.....	10
3.2.3. Herramientas digitales limitadas.....	10
3.2.4. Falta de trazabilidad histórica.....	10
3.2.5. Organización sin indicadores de calidad del modelo	10
3.3. Consideraciones de hardware y software	11
4. Estudio de Factibilidad	11
4.1. Factibilidad Técnica	12
4.2. Factibilidad Económica.....	13

4.2.1. Costos Generales	13
4.2.2. Costos Operativos durante el desarrollo	14
4.2.3. Costos del Ambiente	14
4.2.4. Costos de Personal	15
4.2.5. Costos Totales del Desarrollo del Sistema	15
4.3. Factibilidad Operativa	16
4.4. Factibilidad Legal	17
4.5. Factibilidad Social	17
4.6. Factibilidad Ambiental	18
5. Análisis Financiero	19
5.1. Justificación de la Inversión	19
5.1.1. Beneficios del Proyecto	19
5.1.2. Flujo de Caja Proyectado y Criterios de Inversión	20
5.1.3. Conclusión del Análisis Financiero	22
6. Conclusiones	22

Informe de Factibilidad

1. Descripción del Proyecto

1.1. Nombre del proyecto

DataQuest: Sistema Web de Validación de Normalización de Bases de Datos Relacionales.

1.2. Duración del proyecto

Fecha de Inicio: No especificado en los archivos revisados

Fecha de Fin: No especificado en los archivos revisados

Duración Total: No especificado en los archivos revisados. Para efectos de factibilidad económica se mantiene el horizonte académico de 4 meses utilizado en el modelo FD01.

1.3. Descripción

El proyecto DataQuest es una aplicación web orientada al análisis, diagnóstico y corrección de esquemas de bases de datos relacionales. Su propósito central es apoyar a estudiantes, desarrolladores, administradores de bases de datos y equipos académicos en la verificación de cumplimiento de las formas normales 1FN, 2FN y 3FN, reduciendo errores de diseño lógico antes de implementar una base de datos en un entorno real.

El sistema permite ingresar esquemas mediante sentencias SQL, texto pegado y archivos compatibles, para luego procesarlos mediante algoritmos de normalización. Según los archivos revisados, el análisis se realiza con lógica programada y reglas heurísticas, sin depender de inteligencia artificial. Esta característica permite que los resultados se expliquen de manera trazable y que las sugerencias se vinculen directamente con reglas de normalización, dependencias funcionales y estructura de tablas.

La solución se encuentra desarrollada principalmente en Python. Utiliza Streamlit como interfaz web, FastAPI como servicio de integración para consumo externo, Supabase y PostgreSQL como capa de datos, además de librerías como

sqlparse, pandas, openpyxl, Graphviz, NetworkX y Matplotlib. El repositorio también incluye Dockerfile, archivos de configuración para Terraform y flujos de GitHub Actions orientados a despliegue, pruebas y seguridad.

A nivel funcional, DataQuest incorpora autenticación de usuarios, control de sesión, panel de comunidad, validación de esquemas, generación de diagnóstico, sugerencias de normalización, historial de validaciones, diagramas y endpoint público para diagnóstico de normalización. La base de datos propuesta incluye tablas para perfiles, presencia de usuarios conectados e historial de validaciones, todas integradas con políticas de Row Level Security para restringir el acceso a información propia del usuario.

El valor principal del proyecto consiste en transformar un proceso técnico que normalmente se realiza de forma manual en una experiencia guiada y centralizada. En vez de revisar dependencias funcionales, claves primarias, violaciones y propuestas de corrección de manera aislada, el usuario puede obtener un diagnóstico estructurado, visualizar errores y guardar evidencia de cada análisis dentro del historial del sistema.

1.4. Objetivos

1.4.1. Objetivo general

Implementar una plataforma web para validar la normalización de esquemas de bases de datos relacionales, mediante el análisis de estructuras SQL, detección de violaciones a 1FN, 2FN y 3FN, generación de sugerencias de corrección, visualización de resultados e historial de validaciones, con el fin de mejorar la calidad del diseño lógico de bases de datos.

1.4.2. Objetivos Específicos

- OE1. Analizar esquemas relacionales ingresados por el usuario a través de SQL pegado o archivos compatibles, extrayendo tablas, columnas, claves primarias, claves foráneas y dependencias funcionales cuando estas sean proporcionadas.
- OE2. Detectar automáticamente incumplimientos de Primera, Segunda y Tercera Forma Normal mediante validadores especializados y reglas heurísticas implementadas en la capa core del sistema.
- OE3. Generar sugerencias de corrección que permitan al usuario comprender qué tablas, atributos o dependencias deben separarse, reestructurarse o ajustarse para alcanzar un mayor nivel de normalización.

- OE4. Centralizar la trazabilidad del análisis mediante autenticación, perfiles de usuario, historial de validaciones, almacenamiento de esquemas originales y finales, dependencias, violaciones y sugerencias aplicadas.
- OE5. Facilitar la integración con otros sistemas mediante una API REST desarrollada con FastAPI, capaz de recibir sentencias CREATE TABLE y devolver un diagnóstico estructurado en formato JSON.
- OE6. Mejorar la comprensión académica y técnica de la normalización mediante visualizaciones, diagramas, reportes y una interfaz web orientada a usuarios que requieren validar modelos de base de datos de manera rápida y ordenada.

2. Riesgos

El desarrollo e implementación de DataQuest presenta riesgos técnicos, operativos, académicos, de seguridad y de adopción. Estos riesgos no invalidan el proyecto, pero deben ser gestionados para evitar diagnósticos imprecisos, uso inadecuado de datos, fallas de autenticación o baja confianza por parte de los usuarios. La siguiente matriz resume los principales riesgos identificados y las estrategias de mitigación propuestas.

Riesgo	Probabilidad	Impacto	Estrategia de Mitigación
Diagnósticos incompletos por limitaciones del parser SQL frente a dialectos complejos, constraints avanzadas o sintaxis no contemplada.	Media	Alto	Ampliar progresivamente los casos de prueba, documentar los dialectos soportados, validar errores de sintaxis y mostrar advertencias cuando una sentencia no pueda interpretarse con seguridad.
Uso de resultados automáticos como sustituto de revisión profesional o académica.	Alta	Alto	Mostrar el diagnóstico como apoyo técnico y no como veredicto absoluto. Incluir advertencias sobre revisión docente, DBA o especialista antes de aplicar cambios productivos.
Errores en la detección de dependencias funcionales cuando el usuario no proporciona dependencias explícitas.	Media	Alto	Diferenciar dependencias declaradas por el usuario y dependencias inferidas por heurística. Permitir edición manual de dependencias funcionales antes del diagnóstico final.
Exposición de datos de usuarios, historial o esquemas privados por configuración incorrecta de Supabase o políticas RLS.	Media	Crítico	Mantener Row Level Security, políticas por usuario, variables de entorno protegidas, control de claves API, HTTPS, revisión de permisos y pruebas de acceso por rol.
Indisponibilidad de servicios externos como Supabase, Azure App Service o API pública.	Media	Medio	Implementar manejo de errores, monitoreo básico, respaldos, documentación de despliegue, variables de entorno replicables y pruebas locales mediante Docker.
Baja adopción por usuarios que prefieren revisar normalización manualmente o con herramientas tradicionales.	Media	Medio	Diseñar una interfaz guiada, incluir ejemplos, explicar cada violación, permitir descargar SQL y presentar beneficios concretos en tiempo de revisión y claridad del modelo.
Pérdida de trazabilidad si el historial no guarda correctamente esquemas, niveles, violaciones o sugerencias aplicadas.	Baja	Alto	Validar la tabla historial_validaciones, probar inserciones y actualizaciones, mantener JSON estructurado y respaldos periódicos de PostgreSQL.

3. Análisis de la Situación actual

3.1. Planteamiento del problema

El diseño de bases de datos relacionales requiere aplicar criterios técnicos de normalización para evitar redundancia, anomalías de inserción, actualización y eliminación, dependencias parciales, dependencias transitivas y estructuras difíciles de mantener. Sin embargo, en contextos académicos y de desarrollo inicial, este proceso suele ejecutarse de manera manual, revisando tablas,

claves y atributos con apuntes, hojas de cálculo, diagramas aislados o validación directa por parte del docente o especialista.

Esta situación genera un problema operativo y formativo: el usuario puede construir un esquema aparentemente funcional, pero con deficiencias estructurales que no se detectan hasta etapas posteriores. Cuando una tabla contiene atributos multivaluados, una clave compuesta con dependencias parciales o atributos no clave que dependen de otros atributos no clave, el diseño pierde calidad y aumenta el riesgo de duplicidad de datos, inconsistencias y reprocesos.

La causa principal es la falta de una herramienta centralizada que permita analizar esquemas relacionales de manera guiada, explicar las violaciones encontradas y sugerir mejoras concretas. Aunque existen recursos teóricos sobre normalización, no siempre están integrados a una aplicación que permita cargar un esquema real, generar diagnóstico, visualizar errores, aplicar correcciones y guardar el historial del proceso.

El repositorio de DataQuest responde a esta necesidad mediante una arquitectura web organizada en vistas, controladores, núcleo de análisis y capa de datos. La aplicación permite autenticar usuarios, procesar esquemas, diagnosticar niveles de normalización, mostrar resultados y registrar validaciones en Supabase. Además, el endpoint FastAPI amplía el alcance del sistema al permitir que otros clientes consuman el motor de diagnóstico mediante peticiones HTTP.

No obstante, el README del proyecto indica que varias funcionalidades se encuentran planificadas o pendientes, mientras que el ZIP contiene módulos implementados para interfaz, API, autenticación, diagnóstico, validación, historial y visualización. Por ello, la situación actual puede considerarse como una solución en desarrollo funcional que requiere consolidar pruebas, documentación y validación técnica antes de un despliegue productivo amplio.

3.2. Descripción del árbol de problema

El árbol de problemas del proyecto permite reconocer las causas que originan la validación deficiente de esquemas relacionales. En DataQuest, estas causas se relacionan con revisión manual, ausencia de trazabilidad, falta de herramientas

automáticas, dependencia del criterio individual y escasa integración entre diagnóstico, corrección y almacenamiento del análisis.

3.2.1. Procesos manuales de validación

La revisión de normalización suele realizarse de forma manual, observando cada tabla y sus atributos para identificar si cumple con 1FN, 2FN o 3FN. Esta práctica puede funcionar en ejercicios pequeños, pero se vuelve lenta y propensa a errores cuando el esquema crece, existen claves compuestas o se requiere justificar cada dependencia funcional.

3.2.2. Dependencia del conocimiento individual

El análisis de dependencias funcionales depende en gran medida de la experiencia del estudiante, desarrollador o evaluador. Si el usuario no domina los conceptos de determinante, dependiente, clave candidata o dependencia transitiva, puede aceptar un diseño incorrecto o aplicar una corrección que no resuelve el problema original.

3.2.3. Herramientas digitales limitadas

Aunque existen editores SQL y diagramadores, muchas herramientas no explican directamente si un modelo cumple 1FN, 2FN o 3FN. Tampoco generan sugerencias orientadas al aprendizaje ni registran el proceso completo desde el esquema original hasta el esquema corregido.

3.2.4. Falta de trazabilidad histórica

Cuando el usuario corrige un modelo en varias iteraciones, normalmente pierde evidencia de qué se analizó, qué errores se encontraron, qué sugerencias se aceptaron y qué versión final se obtuvo. Esta ausencia de historial dificulta la retroalimentación académica y la mejora progresiva del diseño.

3.2.5. Organización sin indicadores de calidad del modelo

Sin métricas o reportes, el usuario no puede conocer cuántas validaciones realizó, cuántas tablas fueron analizadas, qué niveles de normalización alcanzó o qué errores se repiten. La falta de indicadores limita la toma de decisiones y el seguimiento del aprendizaje técnico.

3.3. Consideraciones de hardware y software

Se analiza la infraestructura existente y la requerida para ejecutar DataQuest en un entorno académico o piloto. El sistema opera como aplicación web, por lo que los usuarios finales no requieren instalar software especializado en sus equipos; basta con navegador actualizado, conexión a internet y credenciales de acceso.

Hardware Existente (Útil):

- Computadoras personales o laptops de estudiantes, docentes, desarrolladores y usuarios técnicos.
- Conexión a internet para acceder a la aplicación Streamlit, API, Supabase y servicios de despliegue.
- Navegadores modernos como Chrome, Edge o Firefox para utilizar la interfaz web.

Hardware/Software Propuesto (Para la nueva solución):

- Backend y lógica principal: Python 3.10 o superior, módulos core para parser, diagnóstico, validadores 1FN/2FN/3FN, corrector y generador SQL.
- Interfaz web: Streamlit 1.54.0, páginas de autenticación, inicio, comunidad, validador, resultados e historial.
- API de integración: FastAPI y Uvicorn para publicar el endpoint /api/normalizacion y permitir consumo externo del motor de diagnóstico.
- Base de datos: PostgreSQL 16 gestionado mediante Supabase, con tablas internas para perfiles, presencia e historial de validaciones.
- Seguridad y autenticación: Supabase Auth, tokens de sesión, variables de entorno, Row Level Security y políticas de acceso por usuario.
- Visualización: Graphviz, NetworkX, Matplotlib y generación de código Mermaid para representar relaciones y estructura de tablas.
- Despliegue: Dockerfile, GitHub Actions, Terraform y servicios cloud compatibles con Azure App Service o VPS Linux.

4. Estudio de Factibilidad

A continuación se detalla el estudio de factibilidad del proyecto DataQuest, considerando el contenido técnico del ZIP, la arquitectura implementada, los módulos observados, la estructura de base de datos y un escenario académico con proyección de producto digital. Donde no existen montos contractuales reales, se emplean supuestos referenciales para completar el análisis financiero de acuerdo con el formato FD01.

4.1. Factibilidad Técnica

La factibilidad técnica del proyecto es ALTA. El sistema utiliza tecnologías vigentes, de amplia documentación y compatibles con un despliegue web moderno. Python, Streamlit, FastAPI, PostgreSQL, Supabase, Docker y GitHub Actions permiten construir una solución modular, escalable y mantenible sin depender de licencias propietarias obligatorias.

El ZIP evidencia una separación clara entre vista, controlador, lógica de negocio y capa de datos. La vista se implementa con archivos Streamlit en la carpeta views; los controladores orquestan autenticación, comunidad, dashboard y validación; el núcleo core contiene parser, diagnóstico, validadores, corrector, dependencias y generación SQL; finalmente, la carpeta db administra conexión, autenticación, presencia e interacción con Supabase.

La base de datos definida en db/modelos.sql incluye tres tablas principales: perfiles, presencia e historial_validaciones. Estas tablas se relacionan con auth.users de Supabase, permitiendo extender la información del usuario, registrar presencia en tiempo real y almacenar cada validación realizada. Además, el script activa Row Level Security para que cada usuario pueda ver y modificar únicamente su información.

Desde el punto de vista de integración, la existencia de api.py fortalece la factibilidad técnica porque separa el motor de diagnóstico del uso exclusivo de Streamlit. El endpoint POST /api/normalizacion recibe SQL, ejecuta parseo, diagnostica el nivel actual y devuelve violaciones por forma normal, mejoras opcionales y resumen del resultado.

Componente técnico	Situación evaluada	Conclusión
Frontend Streamlit	Interfaz web con páginas para autenticación, inicio, validador, resultados, historial y comunidad.	Adecuado para el alcance académico y funcional del sistema.
Core de normalización	Parser SQL, validadores 1FN/2FN/3FN, diagnóstico, corrector, dependencias y generador SQL.	Viable; requiere ampliar pruebas para dialectos SQL complejos.
Base de datos PostgreSQL/Supabase	Tablas perfiles, presencia e historial_validaciones, con RLS y relación con auth.users.	Sólido para trazabilidad, autenticación e historial por usuario.
API FastAPI	Endpoint público documentado para diagnóstico de normalización mediante JSON.	Escalable para integración con otros sistemas.
DevOps y despliegue	Dockerfile, Terraform, GitHub Actions para pruebas, seguridad y despliegue.	Existe ruta de despliegue reproducible.
Seguridad	Variables de entorno, Supabase Auth, RLS y control de sesiones.	Favorable, pero requiere auditoría antes de producción.

El proyecto es técnicamente factible porque cuenta con arquitectura organizada, tecnologías disponibles, base de datos estructurada, flujo web funcional, endpoint de integración y mecanismos de despliegue. La principal recomendación técnica es consolidar pruebas unitarias, integración, validación de seguridad y documentación de límites del parser antes de una liberación productiva.

4.2. Factibilidad Económica

El análisis económico se centra en los costos que el equipo debe cubrir para completar, probar, documentar y entregar el sistema DataQuest durante un periodo académico estimado de cuatro meses. Al no existir en los archivos revisados un contrato real de pago, se utiliza un presupuesto referencial para mantener la estructura del informe FD01.

4.2.1. Costos Generales

Incluyen materiales e insumos utilizados para documentación, revisión de entregables, evidencias, presentación del proyecto y soporte académico del desarrollo.

Costo de materiales de oficina						
Concepto	Descripción del uso	Tipo de costo	Unidad de estimación	Tiempo	Costo mensual (\$/)	Costo total (\$/)
Papel bond	Utilizado para imprimir borradores, avances, fichas de revisión, actas internas y documentos de trabajo del proyecto.	Consumible	Paquete mensual estimado	4 meses	25.00	100.00
Material de escritura y revisión	Incluye lapiceros, resaltadores, notas adhesivas y marcadores usados para revisión de avances y observaciones técnicas.	Consumible	Conjunto mensual estimado	4 meses	15.00	60.00
Material de organización documental	Incluye folders, clips, grapas, micas y sobres para ordenar entregables, anexos y documentación impresa.	Consumible	Conjunto mensual estimado	4 meses	10.00	40.00
Total materiales de oficina					50.00	200.00

Costo de insumos de impresión						
Concepto	Descripción del uso	Tipo de costo	Unidad de estimación	Tiempo	Costo mensual (\$/)	Costo total (\$/)
Tinta negra	Utilizada para imprimir informes preliminares, documentación técnica, cronogramas y avances del proyecto.	Consumible	Recarga o consumo mensual estimado	4 meses	45.00	180.00
Tinta a color	Utilizada para imprimir diagramas, tablas, evidencias de interfaz, arquitectura y entregables finales.	Consumible	Recarga o consumo mensual estimado	4 meses	35.00	140.00
Total insumos de impresión					80.00	320.00

Resumen de costos generales				
Concepto	Descripción	Costo mensual (S/)	Tiempo	Costo total (S/)
Materiales de oficina	Insumos utilizados para elaboración, revisión, organización y presentación de documentos del proyecto.	50.00	4 meses	200.00
Insumos de impresión	Tinta utilizada para imprimir informes, avances, diagramas, evidencias y entregables finales.	80.00	4 meses	320.00
Total costos generales		130.00	4 meses	520.00

Los costos generales del proyecto ascienden a S/ 520.00, compuestos por S/ 200.00 en materiales de oficina y S/ 320.00 en insumos de impresión. Estos montos son referenciales y se utilizan para mantener consistencia con el formato de evaluación académica.

4.2.2. Costos Operativos durante el desarrollo

Recurso operativo	Tipo	Uso dentro del proyecto	Tiempo	Costo mensual (S/)	Costo total (S/)
Internet de banda ancha	Servicio	Comunicación del equipo, acceso a repositorios, descarga de dependencias, pruebas de Supabase/Azure y envío de entregables.	4 meses	150.00	600.00
Energía eléctrica	Servicio	Funcionamiento de computadoras, router, equipos de red y dispositivos usados para programación, documentación, pruebas y validación.	4 meses	100.00	400.00
Total costos operativos				250.00	1,000.00

Los costos operativos durante el desarrollo ascienden a S/ 1,000.00. Estos servicios permiten sostener el trabajo técnico del equipo, ejecutar pruebas locales, acceder a Supabase, probar la API, mantener comunicación y preparar entregables.

4.2.3. Costos del Ambiente

Recurso del ambiente	Tipo	Uso dentro del proyecto	Tiempo	Costo mensual (S/)	Costo total (S/)
App Service / VPS para Streamlit y API	Servicio cloud	Alojamiento de la interfaz web, API FastAPI, pruebas funcionales y despliegue piloto.	4 meses	160.00	640.00
Supabase PostgreSQL/Auth/Realtime	Backend as a Service	Base de datos, autenticación, políticas RLS, presencia en tiempo real e historial de validaciones.	4 meses	75.00	300.00
Dominio, SSL y monitoreo básico	Servicio digital	Acceso seguro, configuración HTTPS, monitoreo inicial y presentación pública del sistema.	4 meses	30.00	120.00
Almacenamiento y backups	Servicio cloud	Respaldo de datos, evidencias, logs y exportaciones asociadas al sistema.	4 meses	60.00	240.00
Total costos del ambiente				325.00	1,300.00

Los costos del ambiente tecnológico ascienden a S/ 1,300.00. El monto es moderado debido al uso de tecnologías de código abierto, servicios gestionados y despliegue reproducible mediante herramientas incluidas en el repositorio.

4.2.4. Costos de Personal

Rol	Responsable	Función principal	Tiempo	Costo mensual (S/)	Costo total (S/)
Gestor de proyecto / Analista de base de datos	No especificado en archivos revisados	Planificación, análisis de requerimientos, revisión funcional de normalización y control de entregables.	4 meses (80 horas mensuales) (20 horas semanales)	1,200.00 (80 horas x S/ 15.00)	4,800.00
Desarrollador Backend / API	No especificado en archivos revisados	Desarrollo de FastAPI, lógica de diagnóstico, validadores, seguridad de endpoints e integración con Supabase.	4 meses (80 horas mensuales) (20 horas semanales)	1,200.00 (80 horas x S/ 15.00)	4,800.00
Desarrollador Frontend / Streamlit	No especificado en archivos revisados	Desarrollo de vistas, autenticación, validador, resultados, historial, comunidad y experiencia de usuario.	4 meses (80 horas mensuales) (20 horas semanales)	1,200.00 (80 horas x S/ 15.00)	4,800.00
Analista de calidad / Documentador / DevOps	No especificado en archivos revisados	Pruebas, documentación técnica, Docker, GitHub Actions, revisión de seguridad y preparación del informe FD01.	4 meses (80 horas mensuales) (20 horas semanales)	1,200.00 (80 horas x S/ 15.00)	4,800.00
Total costos de personal					19,200.00

Los costos de personal representan el mayor componente del presupuesto, debido a que el desarrollo de DataQuest requiere análisis de normalización, programación Python, interfaz web, API, integración con Supabase, pruebas, documentación y configuración de despliegue. El monto total asciende a S/ 19,200.00.

4.2.5. Costos Totales del Desarrollo del Sistema

Concepto de Costo	Monto Total (S/)
Costos Generales	520.00
Costos Operativos durante el desarrollo	1,000.00
Costos del Ambiente	1,300.00
Costos de Personal	19,200.00
COSTO TOTAL DEL PROYECTO	22,020.00

El costo final estimado del proyecto es de S/ 22,020.00. Para fines de evaluación se considera un pago único referencial por desarrollo e implementación de S/ 28,000.00, cancelado en tres armadas contra entregables: 40% al inicio, 30% a la entrega del prototipo funcional y 30% a la entrega final. Bajo este supuesto, el proyecto genera un margen operativo estimado de S/ 5,980.00 para el equipo de desarrollo.

4.3. Factibilidad Operativa

El sistema DataQuest ha sido diseñado para integrarse al flujo de trabajo de usuarios que requieren validar modelos de bases de datos relacionales antes de implementarlos. La plataforma no reemplaza la revisión académica o profesional, sino que la apoya mediante diagnóstico estructurado, sugerencias de corrección, historial y visualización de resultados.

La capacidad del usuario para operar el sistema es favorable, debido a que la interfaz Streamlit presenta pantallas conocidas: autenticación, inicio, validador, resultados, historial y comunidad. El usuario puede cargar o pegar SQL, analizar el esquema, elegir nivel objetivo, revisar violaciones y descargar o guardar resultados sin instalar software adicional.

El impacto operativo principal se observa en la reducción del tiempo de revisión. En lugar de analizar manualmente cada tabla, identificar dependencias parciales o transitivas y redactar observaciones desde cero, el sistema produce una base de diagnóstico que puede ser revisada y ajustada por el usuario o evaluador.

La capacitación prevista debe ser breve pero práctica. Se recomienda una sesión inicial para estudiantes y usuarios operativos, enfocada en carga de esquemas, lectura de resultados, interpretación de violaciones, uso de historial y consumo del endpoint API. Para administradores, se recomienda capacitación adicional sobre Supabase, políticas RLS, variables de entorno y despliegue.

Lista de Interesados (Stakeholders) y su Impacto:

1. Estudiantes y usuarios académicos: utilizan el sistema para comprender y validar modelos relacionales, reforzando el aprendizaje de normalización.
2. Docentes o evaluadores: reciben evidencia estructurada del análisis realizado, con historial, violaciones y sugerencias aplicadas.
3. Desarrolladores de software: pueden validar esquemas antes de implementarlos, reduciendo errores en etapas tempranas del proyecto.
4. Administrador del sistema: gestiona despliegue, variables de entorno, Supabase, políticas de seguridad, respaldos y monitoreo.
5. Equipos externos o clientes API: consumen el endpoint de normalización para integrar diagnóstico automático en otras plataformas.

Los beneficios operativos tangibles son la centralización del análisis, reducción de errores de diseño, mayor trazabilidad del aprendizaje, generación de

evidencia y posibilidad de integración con otros sistemas. Por ello, la factibilidad operativa es ALTA, siempre que se mantengan instrucciones claras, ejemplos de uso y revisión humana sobre resultados críticos.

4.4. Factibilidad Legal

El proyecto es legalmente factible siempre que su despliegue y uso respeten la protección de datos personales, las licencias de software, los términos de servicios cloud y la naturaleza técnica de sus diagnósticos. DataQuest debe presentarse como herramienta de apoyo para validación de modelos, no como certificador absoluto de calidad de base de datos.

- Protección de Datos Personales: El sistema almacena usuarios, perfiles, presencia e historial de validaciones. Por ello, debe aplicar control de acceso, RLS, políticas de privacidad, variables de entorno seguras y restricción de información por usuario.
- Licencias de software: Python, Streamlit, FastAPI, PostgreSQL, Supabase client, Graphviz, NetworkX y demás dependencias deben utilizarse conforme a sus licencias. El README indica licencia MIT para el proyecto, por lo que debe conservarse el aviso correspondiente si se reutiliza o distribuye el código.
- Uso de servicios externos: Supabase, Azure App Service u otros servicios cloud deben configurarse de acuerdo con sus términos de uso, evitando exposición de claves API o credenciales dentro del repositorio.
- Responsabilidad técnica: Las recomendaciones de normalización deben acompañarse de advertencias, especialmente cuando se basen en inferencias heurísticas. El usuario final debe validar las correcciones antes de aplicarlas en bases de datos productivas.
- Propiedad intelectual: Al no encontrarse en los archivos revisados un contrato de cesión, la titularidad, uso académico, comercialización y mantenimiento deben definirse por escrito antes de entregar el sistema a terceros.

4.5. Factibilidad Social

La implementación de DataQuest tiene impacto social y educativo positivo, debido a que facilita el acceso a una herramienta de análisis de normalización para estudiantes, docentes, desarrolladores junior y usuarios que no cuentan con software especializado. La plataforma transforma conceptos técnicos en resultados comprensibles y accionables.

El sistema contribuye a reducir la brecha entre teoría y práctica. La normalización suele enseñarse mediante ejercicios aislados, pero DataQuest permite cargar

esquemas concretos, detectar fallas y revisar sugerencias sobre modelos reales. Esto favorece el aprendizaje aplicado y la mejora progresiva de competencias técnicas.

La principal barrera social es la confianza en el resultado automático. Para fortalecer la adopción, el sistema debe explicar claramente la causa de cada violación, indicar si una dependencia fue declarada por el usuario o inferida por heurística, y permitir que el usuario tome la decisión final sobre las sugerencias a aplicar.

El proyecto se relaciona con el ODS 4, al fortalecer la educación de calidad mediante herramientas digitales; con el ODS 9, al promover innovación tecnológica; y con el ODS 16, al mejorar trazabilidad, transparencia y control de información dentro de procesos académicos y técnicos.

4.6. Factibilidad Ambiental

El impacto ambiental del proyecto es favorable e indirecto. Al ser una plataforma digital, DataQuest reduce la necesidad de imprimir ejercicios, diagramas, observaciones de revisión, scripts corregidos y reportes de análisis. La evidencia puede mantenerse en formato digital dentro del historial del sistema.

La digitalización del proceso permite disminuir papel, tinta, archivadores y reprocesos asociados a correcciones sucesivas de modelos. Cuando el usuario puede revisar en pantalla el diagnóstico, guardar historial y descargar resultados, se evita imprimir múltiples versiones del mismo esquema.

También se reduce el desplazamiento innecesario para revisiones presenciales, ya que el sistema puede ser utilizado desde navegador y, mediante la API, puede integrarse a plataformas educativas o herramientas de desarrollo. Aunque la infraestructura cloud consume energía, su impacto es moderado frente a los beneficios de centralización y reducción de recursos físicos.

Para fortalecer esta factibilidad se recomienda aplicar política de cero papel, priorizar reportes digitales, dimensionar adecuadamente los servicios cloud, evitar almacenamiento duplicado y programar respaldos racionales.

5. Análisis Financiero

El análisis financiero se aborda desde la óptica de una entidad académica o cliente tecnológico que evalúa contratar el desarrollo e implementación de DataQuest. El objetivo es determinar si el pago único referencial de S/ 28,000.00 se justifica frente a los beneficios operativos y económicos que el sistema puede generar durante una vida útil estimada de tres años.

El costo de oportunidad del capital (COK) se fija en 10% anual, siguiendo un criterio conservador para proyectos tecnológicos de alcance académico. Los beneficios se cuantifican a partir del Año 1, una vez que la plataforma se encuentre operativa. No se consideran beneficios exagerados; se estiman ahorros por reducción de tiempo de revisión, menor reproceso de modelos, centralización de historial y uso de API para automatizar validaciones.

5.1. Justificación de la Inversión

La inversión se justifica porque el sistema atiende un problema concreto: la dificultad de validar modelos relacionales de forma rápida, trazable y comprensible. Un error de normalización puede generar redundancia, inconsistencias, reprocesos y pérdida de tiempo en etapas posteriores de desarrollo. DataQuest reduce este riesgo al entregar diagnóstico y sugerencias desde una interfaz web y una API integrable.

5.1.1. Beneficios del Proyecto

A. Beneficios Tangibles (Cuantificables Anualmente)

A.1. Ahorro en Costos Operativos Directos (Egresos Evitados)

Concepto de Ahorro	Situación Actual (Sin Sistema)	Situación Propuesta (Con DataQuest)	Ahorro Anual Estimado (S/)
Revisión manual de esquemas	Validación de tablas, claves y dependencias mediante revisión manual o documentos separados.	Diagnóstico automatizado de 1FN, 2FN y 3FN desde la interfaz web.	4,680.00
Corrección de errores de normalización	Reprocesos por dependencias parciales, transitivas o atributos mal estructurados.	Sugerencias de corrección y generación de estructura resultante.	3,000.00
Elaboración de reportes y evidencias	Redacción manual de observaciones y capturas dispersas por cada validación.	Historial, resultados estructurados, diagramas y exportación de evidencia.	2,400.00
Centralización de historial técnico	Pérdida de trazabilidad entre esquema original, diagnóstico y versión corregida.	Historial en PostgreSQL con usuario, fecha, niveles, violaciones y sugerencias.	1,920.00
Integración por API	Validaciones repetitivas realizadas manualmente desde otros sistemas o tareas.	Endpoint FastAPI para consumo externo y automatización del diagnóstico.	1,000.00
TOTAL AHORRO OPERATIVO ANUAL			13,000.00

A.2. Incremento en productividad o servicios digitales

Concepto de Beneficio	Detalle del Cálculo	Año 1 (2027)	Año 2 (2028)	Año 3 (2029)
Beneficio incremental por API y uso avanzado	Durante el primer año se considera etapa de estabilización. En los años 2 y 3 se proyecta beneficio por automatización, soporte, API o suscripción institucional de bajo alcance.	0.00	3,000.00	4,500.00
TOTAL BENEFICIO ADICIONAL ANUAL		0.00	3,000.00	4,500.00

Tipo de Beneficio	Año 1 (2027)	Año 2 (2028)	Año 3 (2029)
Ahorro Operativo	S/ 13,000.00	S/ 13,000.00	S/ 13,000.00
Beneficio adicional por productividad/API	S/ 0.00	S/ 3,000.00	S/ 4,500.00
BENEFICIO TOTAL ANUAL	S/ 13,000.00	S/ 16,000.00	S/ 17,500.00

B. Beneficios Intangibles (No Cuantificables, de Alto Valor)

- Mejora del aprendizaje aplicado de normalización mediante diagnósticos explicados.
- Mayor trazabilidad del proceso de validación y corrección de esquemas.
- Reducción de incertidumbre técnica antes de implementar bases de datos reales.
- Mayor estandarización en revisiones académicas o de equipos de desarrollo.
- Posibilidad de integración con otras plataformas mediante API pública documentada.

5.1.2. Flujo de Caja Proyectado y Criterios de Inversión

Concepto	Año 0 (2026)	Año 1 (2027)	Año 2 (2028)	Año 3 (2029)
INGRESOS/BENEFICIOS				
Ahorro Operativo	0.00	13,000.00	13,000.00	13,000.00
Beneficio adicional por productividad/API	0.00	0.00	3,000.00	4,500.00
Total Beneficios	0.00	13,000.00	16,000.00	17,500.00
EGRESOS				
Pago único por desarrollo del sistema DataQuest	28,000.00	0.00	0.00	0.00
Mantenimiento y soporte anual	0.00	1,800.00	1,800.00	1,800.00
Total Egresos	28,000.00	1,800.00	1,800.00	1,800.00
FLUJO DE CAJA NETO	-28,000.00	11,200.00	14,200.00	15,700.00

5.1.2.1. Relación Beneficio/Costo (B/C)

Este indicador compara el valor actual de los beneficios totales con el valor actual de los costos totales. Una relación B/C mayor a 1 indica que, por cada sol invertido, la entidad recibe más de un sol en beneficios actualizados.

Costo (C): Es la suma del valor actual de todos los egresos, considerando inversión inicial más mantenimiento anual.

$$C = 28,000.00 + (1,800.00 / (1.10)^1) + (1,800.00 / (1.10)^2) + (1,800.00 / (1.10)^3)$$

$$C = 28,000.00 + 1,636.36 + 1,487.60 + 1,352.37$$

$$C = S/ 32,476.33$$

Beneficio (B): Es la suma de los beneficios totales traídos a valor presente con el COK del 10%.

$$B = (13,000.00 / (1.10)^1) + (16,000.00 / (1.10)^2) + (17,500.00 / (1.10)^3)$$

$$B = 11,818.18 + 13,223.14 + 13,148.01$$

$$B = S/ 38,189.33$$

$$\text{Relación B/C} = 38,189.33 / 32,476.33 = 1.18$$

Interpretación: El resultado de 1.18 es mayor a 1. Esto indica que, en un horizonte de 3 años, el proyecto es rentable. Por cada Nuevo Sol invertido, la entidad obtiene aproximadamente S/ 1.18 en beneficios actualizados.

5.1.2.2. Valor Actual Neto (VAN)

El VAN calcula la ganancia neta del proyecto en valor actual. Un VAN mayor a 0 significa que el proyecto genera valor económico.

$$VAN = -\text{Inversión Inicial} + \sum (\text{Flujo de Caja Neto} / (1 + \text{COK})^n)$$

Flujo de caja neto considerado: Año 0: -28,000.00; Año 1: 11,200.00; Año 2: 14,200.00; Año 3: 15,700.00.

$$VAN = -28,000.00 + (11,200.00 / 1.10) + (14,200.00 / (1.10)^2) + (15,700.00 / (1.10)^3)$$

$$VAN = -28,000.00 + 10,181.82 + 11,735.54 + 11,795.64$$

$$VAN = S/ 5,713.00$$

Interpretación: El VAN es positivo. Esto confirma que el proyecto recupera la inversión inicial y los costos de soporte, generando valor adicional para la entidad usuaria.

5.1.2.3. Tasa Interna de Retorno (TIR)

La TIR es la tasa de descuento que hace que el VAN sea igual a cero. Representa la rentabilidad porcentual del proyecto y se acepta si es superior al COK del 10%.

$$0 = -28,000.00 + (11,200.00 / (1+TIR)^1) + (14,200.00 / (1+TIR)^2) + (15,700.00 / (1+TIR)^3)$$

$$TIR \approx 20.60\%$$

Interpretación: La TIR del 20.60% es superior al COK del 10%, por lo que la rentabilidad proyectada es adecuada para un proyecto tecnológico de alcance académico con proyección de servicio digital.

5.1.3. Conclusión del Análisis Financiero

El análisis financiero demuestra la viabilidad del proyecto DataQuest bajo supuestos conservadores. La inversión única de S/ 28,000.00 y el mantenimiento anual de S/ 1,800.00 se justifican por beneficios tangibles crecientes durante tres años. Los indicadores B/C = 1.18, VAN = S/ 5,713.00 y TIR = 20.60% son positivos y superan los umbrales de aceptación, incluso sin monetizar beneficios intangibles como aprendizaje aplicado, trazabilidad, reducción de errores y automatización de revisiones técnicas.

6. Conclusiones

- El proyecto es técnicamente factible. El ZIP evidencia una arquitectura organizada con Python, Streamlit, FastAPI, Supabase, PostgreSQL, Docker, Terraform y GitHub Actions, además de módulos diferenciados para vista, controlador, lógica de normalización y capa de datos.
- El proyecto es económicamente factible para el equipo de desarrollo. El costo total estimado asciende a S/ 22,020.00; bajo un pago referencial de S/ 28,000.00, existe margen operativo positivo para cubrir personal, ambiente tecnológico, servicios y documentación.
- El proyecto es financieramente viable para la entidad usuaria. Con un VAN positivo de S/ 5,713.00, una relación B/C de 1.18 y una TIR de 20.60%, la inversión se recupera en el horizonte evaluado y genera valor neto.
- El proyecto es operativamente factible. La plataforma centraliza autenticación, validador de esquemas, diagnóstico de normalización, sugerencias, historial, comunidad y API, reduciendo la dispersión de herramientas y mejorando la trazabilidad del análisis.
- El proyecto es legalmente factible si se mantiene como herramienta de apoyo técnico, se protegen los datos de usuarios, se respetan licencias de software, se preservan claves de entorno y se documentan claramente los límites de los diagnósticos heurísticos.

- El proyecto es socialmente pertinente porque facilita el aprendizaje de normalización y el acceso a una herramienta de validación para estudiantes, docentes y desarrolladores, fortaleciendo competencias en diseño de bases de datos relacionales.
- El proyecto presenta impacto ambiental favorable al promover revisión digital, historial en línea, reducción de impresiones y menor uso de documentos físicos durante la validación de modelos.
- En conclusión general, DataQuest es técnica, económica, financiera, operativa, legal, social y ambientalmente factible como sistema web de apoyo para la validación de normalización de bases de datos relacionales. Su implementación debe acompañarse de pruebas ampliadas, documentación de límites, capacitación y control de seguridad antes de su uso productivo.